San Jose State University
CS 147 - Computer Architecture
Class Project - Phase 1

Overall Project Summary

The CS-147 term project is a complete implementation of the WISC-SJ26 microprocessor. All components of your design will be written in Verilog. Like the course homework assignments, your Verilog code is expected to pass all Vcheck tests. Code that passes all logic tests but fails the Vcheck test will still be considered failing code.

Each stage of the design makes the processor progressively more complicated. For your own benefit, it is strongly recommended that you not proceed to a new stage before you are confident the current stage is working to specification. Debugging errors in a complex design is much harder. It is almost always better to test smaller, simpler components first. Many of the Verilog problems in the homework assignments were designed to be compatible with the project. Please feel free to reuse these modules (of course, fixing any errors first!).

An assembler for the WISC-SJ26 ISA is provided for your use. Sample test programs are also provided, although you are strongly encouraged to write custom tests to augment these. Be aware that these test programs were written for a slightly different ISA specification and therefore may not work as advertised. It will be your job as diligent designers to determine if unexpected behavior occurs due to a bug in your design or as the result of the change in ISA. A syntax guide for interacting with the assembler can be found in the commands section of this document.

Overall Timeline and Grade Distribution

Due Date	**Project Component**	
2/26/26	Design Review	(5% of project grade)
3/10/26	Phase #1	(15% of project grade)
4/7/26	Phase #2	(30% of project grade)
4/16/26	Phase #2.1	(10% of project grade)
N/A	Phase #2.2	(Optional)
4/23/26	Phase #2.3 Caching	(10% of project grade)
5/7/26	Phase #3	(30% of project grade)

Design Review

Each student should create a complete schematic of an unpipelined WISC-SJ26 implementation. This schematic should ideally be completed on a digital markup tool, such as draw.io. Hand drawn diagrams will be accepted, but are not preferred. Each module, bus, and signal should be uniquely labeled. The schematic should be hierarchical so that the top-level design contains only empty shells for each planned submodule. In general, there will be a one-to-one mapping of modules in your schematic to the modules you will eventually write in Verilog. The textbook pipeline diagram(s) is a good starting point but there are many differences between it and the ISA for this project. You will need to look at the class ISA and make sure to adapt it for that.

Your schematic should explicitly include modules for the Fetch, Decode, Execute, Memory, and Writeback, and the code for these modules will go into the provided shell modules. While explicitly drawing the pipeline stages that correspond to these modules is not required in the schematic, you should still design with a pipeline in mind. It is a good idea to place modules near their final location in the pipelined design.

During the review, individual team members should be able to describe the datapath of any legal WISC-SJ26 instruction using the schematic as a reference. Teams will also be expected to defend the design decisions that they make. You need to have thought through the control path and decode logic. It is not necessary to have done a complete table of signals, but if you have such a table with the control signal values for every instruction, that would be great.

Your design should be submitted to Canvas for review and grading.

Phase 1 Overview

In Phase 1 of the project, you will implement a single-cycle, non-pipelined WISC-SJ26 instruction-set-compliant processor. The following textbook figures are useful references for a high-level overview of processor structure:

- Figure 4.33 (p. 299)
- Figure 4.35 (p. 301)
- Figure 4.36 (p. 303)

For this stage, use the single-cycle perfect memory model. Because instruction fetch and data access are both required, instantiate two memories: one instruction memory and one data memory.

Your design must support the full WISC-SJ26 instruction set, excluding extra credit instructions (see `./assignments/project/common/writeup/wisc-sj26_spec.pdf`). Your implementation must use the single-cycle memory model.

You are provided with stub files to bootstrap implementation:

- `fetch.v`
- `decode.v`
- `execute.v`
- `memory.v`
- `wb.v`
- `regFile.v`
- `proc.v`

Additionally, you may reuse modules from previous homework assignments. If you reuse a module, copy the full module source into `./assignments/project/demo1/verilog`. Do not link to files in other directories, as cross-directory linking can cause toolchain and grading inconsistencies.

Commands

Testing

To run the built in testing suite, run:

```
./run test project phase_1
```

Note that this can take a significant amount of time, especially if you are on a slower computer, so you should plan ahead accordingly. If you're submitting this project at the last minute (which you shouldn't do in general), set aside at minimum thirty minutes to allow all tests to run to completion, and to allow you to upload to Gradescope.

You can also run individual tests if you are only failing a few. To do this, add the name of the desired test to the end of the command, for example:

```
./run test project phase_1 add_0
```

to re-run the `add_0` test only.

To run verbose output, run with the `-v` argument:

```
./run test project phase_1 <subtest> -v
```

You are advised to only run verbose mode with individual tests, as the output from the non-verbose test is already extremely long, but this feature is available if you need it.

Custom tests should be placed in `./assignments/project/testprograms/student_custom`. Add each custom test filename to `./assignments/project/testprograms/student_custom/all.list`, one filename per line. Once listed, custom tests can be run with:

```
./run test project phase_1 <subtest>
```

using the filename without the `.asm` extension.

Submitting

To generate your submission file, use:

```
./run build_submission project phase_1
```

This will run the full testing suite (remember to set aside a significant amount of time), and run your submission through the integrated autograder. Your submission file will then be dropped in:

```
./generated_turnins/project_phase_1
```

Upload this generated zip file to Gradescope in order to submit.

Assembing Custom Tests

1. Create custom tests by implementing `.asm` files with your desired test, and placing them into: `./assignments/project/testprograms/student_custom`
2. Assemble your custom tests with one of the following commands:

   ```
   ./run assemble student_custom
   ```

   ```
   ./run assemble student_custom my_test.asm
   ```

   The first form assembles all `.asm` files in `./student_custom`. The second form assembles only the named test file (which must exist in `./student_custom` and include the `.asm` extension). Generated `.img` files are written to the `./assembled` subfolder of the `./student_custom` directory.

3. Add the tests you wish to include in the full testing suite to: `./assignments/project/testprograms/student_custom/all.list` Add one filename per line, including the `.asm` extension (for example: `my_test.asm`). These tests will run alongside the integrated testing suite as the first testing group. Tests can be run individually without adding them to the testing suite.

4. Once tests have been assembled, you can run them individually via the same syntax used in the Testing section, or run the full suite. To remove a custom test from the full suite, remove its filename, or comment the relevant line from `all.list`.

NOTE: If you have a significant number of custom tests, you will be prompted for confirmation prior to assembly.